

CEO GENAI Analysis

- a.) I used Chatgpt and Gemini
- b.) I accessed these GenAi using my laptop on Chrome.

Assignment:

1. Use two GenAI's to generate C code to implement the following requirements (Note: You may use either ChatGPT or CoPilot as one of your choices, but not both. They are essentially the same GenAI):
 - The program will prioritize emails for a busy company CEO.
 - You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
 - You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
 - You must create functions from scratch. Do not include pre-existing heap modules.

c.)

• Sample Test File:

```
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT
```

• Your program should display this:

There are 6 emails to read.

Next email:

Sender: Boss
Subject: Never Mind
Date: 01-03-2025

Next email:

Sender: Boss
Subject: Important
Date: 12-20-2024

There are 4 emails to read.

• Test File format:

Command	Description
EMAIL <sender category>,<subject line>,<date>	The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows: • Boss – read first • Subordinate – read next • Peer – read next • ImportantPerson – read next • OtherPerson – read last If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint. EMAIL is followed by space. The rest of the fields are delimited. Assume <sender category> is one of the five strings listed above. Assume <subject line> is a string which may contain spaces, but not commas Assume <date> is in the format: MM-DD-YYYY
NEXT	Next email for the CEO to read. Display the information on the terminal in the following format: Sender: Subject: Date:
READ	CEO has read email and dealt with it
COUNT	Display current count of unread emails

- The grader will test your code with a different test file. The test file will begin with the emails in the CEO's Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

I simply used screen shots of the provided examples and prompts. As well as a prompt asking the ai to explain the way it used time complexity in its code. I also made sure to give both ai's the exact same prompts so that one couldn't have any possibility of having some sort of advantage.

e.)

Correctness -

Chatgpt: Works correctly, gives user a prompt to enter the .txt file in.

Gemini: Works correctly, does not give user a prompt, instead you must enter, `“./geminiAnalysis < test1.txt”` in the terminal. This formatting can cause a problem confusing to users who do not have much experience with programming.

Execution Time: Both generated codes have the same execution times.

Space Complexity -

Chatgpt: A smaller space compacity that uses iteration to bubble down.

Gemini: A larger space compacity that uses recursion to bubble down.

Maintainability -

Chatgpt: Uses much more comments in its coding to explain the functions. Also about twice as long as Gemini. Can check for failure. Simply returns numbers and does not notify where values come from.

Gemini: Uses no comments. The shorter coding can make it much faster to read though. Does not have a way to check for failure. Has named constraints.

f.) I decided to use Chatgpt as my final code. The comments make reading through the program easier, as well as how the functions are not completely coded together and instead separated. The separated functions are also easier to modify if and when needed.

d.) The next pages will be the entire codes that were generated

Chatgpt

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#define MAX_SUBJECT 500
```

```
#define MAX_LINE 600
```

```
/* -----
```

```
Email structure
```

```
----- */
```

```
typedef struct {
```

```
    char sender[30];
```

```
    char subject[MAX_SUBJECT];
```

```
    int month;
```

```
    int day;
```

```
    int year;
```

```
    int priority;
```

```
} Email;
```

```
/* -----
```

```
MaxHeap structure
```

```
----- */  
  
typedef struct {  
    Email *emails;  
  
    int size;  
  
    int capacity;  
} MaxHeap;
```

```
/* -----
```

Get priority for sender category

Lower number = higher priority.

Boss is highest priority.

```
----- */  
  
int getPriority(char sender[]) {  
    if (strcmp(sender, "Boss") == 0)  
        return 5;  
  
    if (strcmp(sender, "Subordinate") == 0)  
        return 4;  
  
    if (strcmp(sender, "Peer") == 0)  
        return 3;  
  
    if (strcmp(sender, "ImportantPerson") == 0)  
        return 2;
```

```

    if (strcmp(sender, "OtherPerson") == 0)
        return 1;

    return 0;
}

/* -----
   Convert date to a number so dates can be compared.

   A larger value means a newer date.
   ----- */
int dateValue(Email *email) {
    return email->year * 10000 +
        email->month * 100 +
        email->day;
}

/* -----
   Compare two emails.

   Returns 1 if a should appear BEFORE b in the priority
   queue.

   Priority category is considered first.
   If categories are equal, newest date comes first.
   ----- */

```

```

int higherPriority(Email *a, Email *b) {

    if (a->priority > b->priority)
        return 1;

    if (a->priority < b->priority)
        return 0;

    /* Same sender category:
       newest email goes first */
    if (dateValue(a) > dateValue(b))
        return 1;

    return 0;
}

/* -----
   Initialize heap
   ----- */

void initializeHeap(MaxHeap *heap) {
    heap->capacity = 10;
    heap->size = 0;

    heap->emails = malloc(heap->capacity * sizeof(Email));

    if (heap->emails == NULL) {

```

```

        printf("Memory allocation failed.\n");
        exit(1);
    }
}

/* -----
   Resize heap when it becomes full
   ----- */
void resizeHeap(MaxHeap *heap) {
    heap->capacity *= 2;

    Email *temp = realloc(
        heap->emails,
        heap->capacity * sizeof(Email)
    );

    if (temp == NULL) {
        printf("Memory allocation failed.\n");
        free(heap->emails);
        exit(1);
    }

    heap->emails = temp;
}

/* -----

```

Swap two emails

----- */

```
void swapEmails(Email *a, Email *b) {
```

```
    Email temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

/* -----

Insert an email into the MaxHeap

----- */

```
void insertHeap(MaxHeap *heap, Email email) {
```

```
    if (heap->size == heap->capacity) {
```

```
        resizeHeap(heap);
```

```
    }
```

/ Put new email at the end */*

```
    int index = heap->size;
```

```
    heap->emails[index] = email;
```

```
    heap->size++;
```

/ Bubble up */*

```
    while (index > 0) {
```

```
        int parent = (index - 1) / 2;
```



```
if (!higherPriority(  
    &heap->emails[index],  
    &heap->emails[parent])) {  
    break;  
}
```

```
swapEmails(  
    &heap->emails[index],  
    &heap->emails[parent]  
);
```

```
    index = parent;  
}  
}
```

```
/* -----
```

Return the highest-priority email.

This does NOT remove the email.

```
----- */
```

```
Email *peekHeap(MaxHeap *heap) {
```

```
    if (heap->size == 0) {  
        return NULL;  
    }
```

```
    return &heap->emails[0];  
}
```

```
/* -----
```

```
Remove the highest-priority email.
```

```
Used by READ.
```

```
----- */
```

```
Email removeHeap(MaxHeap *heap) {
```

```
    Email removed = heap->emails[0];
```

```
    /* Move last item to root */
```

```
    heap->size--;
```

```
    if (heap->size > 0) {
```

```
        heap->emails[0] =
```

```
            heap->emails[heap->size];
```

```
        /* Bubble down */
```

```
        int index = 0;
```

```
        while (1) {
```

```
int left = 2 * index + 1;
```

```
int right = 2 * index + 2;
```

```
int largest = index;
```

```
if (left < heap->size &&
```

```
    higherPriority(
```

```
        &heap->emails[left],
```

```
        &heap->emails[largest])) {
```

```
    largest = left;
```

```
}
```

```
if (right < heap->size &&
```

```
    higherPriority(
```

```
        &heap->emails[right],
```

```
        &heap->emails[largest])) {
```

```
    largest = right;
```

```
}
```

```
if (largest == index) {
```

```
    break;
```

```
}
```

```
swapEmails(
```

```
    &heap->emails[index],
```

```

        &heap->emails[largest]

    );

    index = largest;
}
}

return removed;
}

/* -----
   Display an email in the required format
   ----- */
void displayEmail(Email *email) {

    printf("Next email:\n");
    printf("  Sender: %s\n", email->sender);
    printf("  Subject: %s\n", email->subject);

    printf("  Date: %02d-%02d-%04d\n",
        email->month,
        email->day,
        email->year);
}

/* -----

```

Parse an EMAIL command.

Format:

EMAIL <sender category>,<subject>,<date>

Example:

EMAIL Boss,Important,12-20-2024

*----- */*

```
int parseEmail(char *line, Email *email) {
```

```
    char *contents;
```

```
    char *comma1;
```

```
    char *comma2;
```

```
    /* Find "EMAIL " */
```

```
    contents = line + 6;
```

```
    /* Remove newline */
```

```
    contents[strcspn(contents, "\n")] = '\0';
```

```
    /* Find commas */
```

```
    comma1 = strchr(contents, ',');
```

```
    if (comma1 == NULL)
```

```
        return 0;
```

```
comma2 = strchr(comma1 + 1, ',');
```

```
if (comma2 == NULL)
```

```
    return 0;
```

```
/* Sender */
```

```
*comma1 = '\0';
```

```
strcpy(email->sender, contents);
```

```
/* Subject */
```

```
*comma2 = '\0';
```

```
strcpy(email->subject, comma1 + 1);
```

```
/* Date */
```

```
if (sscanf(
```

```
    comma2 + 1,
```

```
    "%d-%d-%d",
```

```
    &email->month,
```

```
    &email->day,
```

```
    &email->year
```

```
) != 3) {
```

```
    return 0;
```

```
}
```

```
email->priority = getPriority(email->sender);
```

```

    return 1;
}

/* -----
   Main
   ----- */

int main(void) {

    char filename[100];
    char line[MAX_LINE];

    MaxHeap heap;

    initializeHeap(&heap);

    /* Get test file name */
    printf("Enter test file name: ");
    scanf("%99s", filename);

    FILE *file = fopen(filename, "r");

    if (file == NULL) {
        printf("Could not open file.\n");
        free(heap.emails);
        return 1;
    }

```

```
}
```

```
/*-----
```

```
Read commands from file
```

```
----- */
```

```
while (fgets(line, sizeof(line), file) != NULL) {
```

```
/* Remove newline */
```

```
line[strcspn(line, "\n")] = '\0';
```

```
/* EMAIL command */
```

```
if (strncmp(line, "EMAIL ", 6) == 0) {
```

```
    Email email;
```

```
    if (parseEmail(line, &email)) {
```

```
        insertHeap(&heap, email);
```

```
    }
```

```
}
```

```
/* NEXT command */
```

```
else if (strcmp(line, "NEXT") == 0) {
```

```
    Email *email = peekHeap(&heap);
```

```
    if (email != NULL) {
```



```
        displayEmail(email);
    }
}

/* READ command */
else if (strcmp(line, "READ") == 0) {

    if (heap.size > 0) {
        removeHeap(&heap);
    }
}

/* COUNT command */
else if (strcmp(line, "COUNT") == 0) {

    printf("There are %d emails to read.\n",
        heap.size);
}

fclose(file);
free(heap.emails);

return 0;
}
```

Gemini

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
// Priority constants
```

```
#define PRIORITY_BOSS 5
```

```
#define PRIORITY_SUBORDINATE 4
```

```
#define PRIORITY_PEER 3
```

```
#define PRIORITY_IMPORTANT 2
```

```
#define PRIORITY_OTHER 1
```

```
#define PRIORITY_UNKNOWN 0
```

```
typedef struct {
```

```
    char sender[50];
```

```
    char subject[200];
```

```
    char date[15];
```

```
    int priority_level;
```

```
    long date_val;
```

```
    int insert_id; // Tie-breaker for same-day emails (newer = inserted later)
```

```
} Email;
```

```
typedef struct {
```

```
    Email* array;
```

```

    int size;

    int capacity;
} MaxHeap;

// -----
// Helper Functions for Parsing & Prioritization
// -----

int get_priority(const char* sender) {
    if (strcmp(sender, "Boss") == 0) return PRIORITY_BOSS;
    if (strcmp(sender, "Subordinate") == 0) return PRIORITY_SUBORDINATE;
    if (strcmp(sender, "Peer") == 0) return PRIORITY_PEER;
    if (strcmp(sender, "ImportantPerson") == 0) return PRIORITY_IMPORTANT;
    if (strcmp(sender, "OtherPerson") == 0) return PRIORITY_OTHER;
    return PRIORITY_UNKNOWN;
}

// Converts MM-DD-YYYY to an integer YYYYMMDD for easy comparison
long parse_date(const char* date_str) {
    int m, d, y;

    if (sscanf(date_str, "%d-%d-%d", &m, &d, &y) == 3) {
        return (long)y * 10000 + m * 100 + d;
    }

    return 0;
}

```

// Returns >0 if a has higher priority than b, <0 if lower, 0 if equal

```
int compare_emails(Email a, Email b) {  
    if (a.priority_level != b.priority_level) {  
        return a.priority_level - b.priority_level;  
    }  
    if (a.date_val != b.date_val) {  
        return (a.date_val > b.date_val) ? 1 : -1;  
    }  
    return a.insert_id - b.insert_id;  
}
```

// -----

// MaxHeap Implementation (From Scratch)

// -----

```
void init_heap(MaxHeap* heap, int initial_capacity) {  
    heap->capacity = initial_capacity;  
    heap->size = 0;  
    heap->array = (Email*)malloc(sizeof(Email) * heap->capacity);  
}
```

```
void free_heap(MaxHeap* heap) {  
    free(heap->array);  
    heap->size = 0;  
    heap->capacity = 0;  
}
```

```

void heapify_up(MaxHeap* heap, int index) {
    while (index > 0) {
        int parent_index = (index - 1) / 2;
        if (compare_emails(heap->array[index], heap->array[parent_index]) > 0) {
            Email temp = heap->array[index];
            heap->array[index] = heap->array[parent_index];
            heap->array[parent_index] = temp;
            index = parent_index;
        } else {
            break;
        }
    }
}

```

```

void heapify_down(MaxHeap* heap, int index) {
    int max_index = index;
    int left_child = 2 * index + 1;
    int right_child = 2 * index + 2;

    if (left_child < heap->size && compare_emails(heap->array[left_child], heap->array[max_index]) > 0) {
        max_index = left_child;
    }

    if (right_child < heap->size && compare_emails(heap->array[right_child], heap->array[max_index]) > 0) {
        max_index = right_child;
    }
}

```

```
}
```

```
if (max_index != index) {  
    Email temp = heap->array[index];  
    heap->array[index] = heap->array[max_index];  
    heap->array[max_index] = temp;  
    heapify_down(heap, max_index);  
}  
}
```

```
void insert_email(MaxHeap* heap, Email email) {  
    if (heap->size == heap->capacity) {  
        heap->capacity *= 2;  
        heap->array = (Email*)realloc(heap->array, sizeof(Email) * heap->capacity);  
    }  
    heap->array[heap->size] = email;  
    heapify_up(heap, heap->size);  
    heap->size++;  
}
```

```
void extract_max(MaxHeap* heap) {  
    if (heap->size == 0) return;  
  
    // Replace root with the last element  
    heap->array[0] = heap->array[heap->size - 1];  
    heap->size--;
```

```
if (heap->size > 0) {
    heapify_down(heap, 0);
}
```

```
// -----  
// Main Execution  
// -----
```

```
int main() {  
    MaxHeap inbox;  
    init_heap(&inbox, 100); // Start with an arbitrary reasonable capacity  
  
    char line[512];  
    int global_insert_id = 0;  
  
    // Read commands line by line from stdin (or a redirected file)  
    while (fgets(line, sizeof(line), stdin)) {  
        // Strip trailing newlines  
        line[strcspn(line, "\r\n")] = 0;  
  
        if (strlen(line) == 0) continue;  
  
        if (strncmp(line, "EMAIL ", 6) == 0) {  
            char* payload = line + 6; // Skip "EMAIL "
```

```

char* sender = strtok(payload, ",");
char* subject = strtok(NULL, ",");
char* date = strtok(NULL, ",");

if (sender && subject && date) {
    Email new_email;

    strncpy(new_email.sender, sender, sizeof(new_email.sender) - 1);
    new_email.sender[sizeof(new_email.sender) - 1] = '\0';

    strncpy(new_email.subject, subject, sizeof(new_email.subject) - 1);
    new_email.subject[sizeof(new_email.subject) - 1] = '\0';

    strncpy(new_email.date, date, sizeof(new_email.date) - 1);
    new_email.date[sizeof(new_email.date) - 1] = '\0';

    new_email.priority_level = get_priority(new_email.sender);
    new_email.date_val = parse_date(new_email.date);
    new_email.insert_id = ++global_insert_id;

    insert_email(&inbox, new_email);
}
}

else if (strcmp(line, "NEXT") == 0) {
    if (inbox.size > 0) {
        Email top_email = inbox.array[0];

```



```
    printf("Next email:\n");

    printf("  Sender: %s\n", top_email.sender);
    printf("  Subject: %s\n", top_email.subject);
    printf("  Date: %s\n\n", top_email.date);
}
}

else if (strcmp(line, "READ") == 0) {
    extract_max(&inbox);
}

else if (strcmp(line, "COUNT") == 0) {
    printf("There are %d emails to read.\n\n", inbox.size);
}
}

free_heap(&inbox);

return 0;
}
```